

## PROYECTO GUÍA FULLSTACK

### Technical Support Ticket Management System

#### MÓDULO 3. Modelado del Dominio del Sistema de Tickets

##### Objetivos:

- Diseñar las entidades que representarán la lógica de los tickets y sus relaciones con los usuarios de Identity.
- Definir la lógica de datos del negocio usando C# 14.
- Migración a SQL Server

##### Contenido:

- Creación de Enums: `TicketStatus` (Open, InProgress, Resolved, Closed), `TicketPriority`.
- Clases de Entidad: `Ticket`, `Category`, `TicketComment`.
- Relaciones: Un `Ticket` tiene un `Author` (User) y puede tener un `Technician` asignado.
- Uso de *Primary Constructors* y *Required members* de C# 14.

**Resultado:** Migración aplicada con las tablas de negocio relacionadas a la tabla de usuarios.

#### 1. Definición de Enums (Los Estados del Negocio)

Se definen estados fijos. Crear una carpeta llamada `/Enums` en la raíz del proyecto.

**Archivo:** `/Enums/TicketStatus.cs`

```
15 references
3      public enum TicketStatus
4      {
5          Open,
6          InProgress,
7          Resolved,
8          Closed
9      }
10 }
```

**Archivo:** `/Enums/TicketPriority.cs`

```
4 references
3      public enum TicketPriority
4      {
5          Low,
6          Medium,
7          High,
8          Urgent
9      }
10 }
```

## 2. Creación de las Entidades de Negocio

Crea una carpeta llamada `/Models` en la raíz del proyecto. Aquí se definen las principales clases para el proyecto.

**Archivo:** `/Models/Category.cs` (Uso de `required` C# 14)

```

3      7 references
4      public class Category
5      {
6          1 reference
7          public int Id { get; set; }
8          5 references
9          public required string Name { get; set; }
10         3 references
11         public string? Description { get; set; }
12     }

```

**Archivo:** `/Models/Ticket.cs` (El centro del sistema)

Notar que se relaciona el ticket con `ApplicationUser` para saber quién lo creó y quién lo atiende.

```

using TicketsSystem.Data;
using TicketsSystem.Enums;

```

```

6      14 references
7      public class Ticket
8      {
9          4 references
10         public int Id { get; set; }
11
12         8 references
13         public required string Title { get; set; }
14         6 references
15         public required string Description { get; set; }
16
17         2 references
18         public DateTime CreatedAt { get; set; } = DateTime.UtcNow;
19         2 references
20         public DateTime? UpdatedAt { get; set; }
21
22         9 references
23         public TicketStatus Status { get; set; } = TicketStatus.Open;
24         6 references
25         public TicketPriority Priority { get; set; } = TicketPriority.Medium;
26
27         // Relación con Categoría
28         5 references
29         public int CategoryId { get; set; }
30         4 references
31         public Category? Category { get; set; }
32
33         // Relación con Usuarios (Identity)
34         4 references
35         public required string AuthorId { get; set; }
36         6 references
37         public ApplicationUser? Author { get; set; }
38
39         5 references
40         public string? TechnicianId { get; set; }
41         6 references
42         public ApplicationUser? Technician { get; set; }
43
44         // Colección de comentarios
45         0 references
46         public ICollection<TicketComment> Comments { get; set; } = new List<TicketComment>();
47     }

```

Archivo: /Models/TicketComment.cs

```
3      3 references
4      public class TicketComment
5      {
6          0 references
7          public int Id { get; set; }
8          0 references
9          public required string Content { get; set; }
10         0 references
11         public DateTime CreatedAt { get; set; } = DateTime.UtcNow;
12
13         0 references
14         public int TicketId { get; set; }
15         0 references
16         public Ticket? Ticket { get; set; }
17
18         0 references
19         public required string UserId { get; set; }
20         0 references
21         public Data.ApplicationUser? User { get; set; }
22     }
```

Actualizando el repositorio:

```
Developer PowerShell
+ Developer PowerShell v | | |
PS C:\Users\luisd\source\repos\SupportTicketsSystem2026> git add .
PS C:\Users\luisd\source\repos\SupportTicketsSystem2026> git status
On branch main
Your branch is up to date with 'origin/main'.

Changes to be committed:
  (use "git restore --staged <file>..." to unstage)
    new file:   SupportTicketsSystem2026/Enums/TicketPriority.cs
    new file:   SupportTicketsSystem2026/Enums/TicketStatus.cs
    new file:   SupportTicketsSystem2026/Models/Category.cs
    new file:   SupportTicketsSystem2026/Models/Ticket.cs
    new file:   SupportTicketsSystem2026/Models/TicketComment.cs

PS C:\Users\luisd\source\repos\SupportTicketsSystem2026>

PS C:\Users\luisd\source\repos\SupportTicketsSystem2026> git commit -m "Implementando las entidades de la app: ticket, status, comment, category... MÓDULO 3"
[main 8ccf996] Implementando las entidades de la app: ticket, status, comment, category... MÓDULO 3
5 files changed, 30 insertions(+)
create mode 100644 SupportTicketsSystem2026/Enums/TicketPriority.cs
create mode 100644 SupportTicketsSystem2026/Enums/TicketStatus.cs
create mode 100644 SupportTicketsSystem2026/Models/Category.cs
create mode 100644 SupportTicketsSystem2026/Models/Ticket.cs
create mode 100644 SupportTicketsSystem2026/Models/TicketComment.cs
PS C:\Users\luisd\source\repos\SupportTicketsSystem2026>

PS C:\Users\luisd\source\repos\SupportTicketsSystem2026> git push origin main
Enumerating objects: 12, done.
Counting objects: 100% (12/12), done.
Delta compression using up to 24 threads
Compressing objects: 100% (10/10), done.
Writing objects: 100% (10/10), 990 bytes | 990.00 KiB/s, done.
Total 10 (delta 3), reused 0 (delta 0), pack-reused 0 (from 0)
remote: Resolving deltas: 100% (3/3), completed with 2 local objects.
To https://github.com/siciuprp2020/SupportTicketsSystem2026.git
 23f8cfc..8ccf996  main -> main
PS C:\Users\luisd\source\repos\SupportTicketsSystem2026>
```

### 3. Integración en el ApplicationDbContext

Ahora debemos "avisarle" a Entity Framework que estas clases deben convertirse en tablas. Abrir /Data/ApplicationDbContext.cs. Agregar el siguiente código:

```
8      {
9          // Aquí registraremos los DbSet en el Módulo 3
10         public DbSet<Ticket> Tickets { get; set; }
11         public DbSet<Category> Categories { get; set; }
12         public DbSet<TicketComment> TicketComments { get; set; }
13
14         protected override void OnModelCreating(ModelBuilder builder)
15         {
16             base.OnModelCreating(builder);
17
18             // Configuración adicional si fuera necesaria (Fluent API)
19             builder.Entity<Ticket>()
20                 .HasOne(t => t.Author)
21                 .WithMany()
22                 .HasForeignKey(t => t.AuthorId)
23                 .OnDelete(DeleteBehavior.Restrict);
24
25             builder.Entity<Ticket>()
26                 .HasOne(t => t.Technician)
27                 .WithMany()
28                 .HasForeignKey(t => t.TechnicianId)
29                 .OnDelete(DeleteBehavior.Restrict);
30         }
31     }
```

La **Fluent API** en Entity Framework Core se utiliza para configurar el mapeo entre las clases de C# y las tablas de SQL Server cuando las **convenciones por defecto** o las **Data Annotations** (atributos como [Key], [Required]) no son suficientes o no son deseadas por razones de arquitectura.

Aquí se detallan los escenarios principales donde es necesaria o muy recomendable:

#### 1. Relaciones Múltiples hacia la Misma Entidad

Es el motivo técnico más frecuente. Si una entidad tiene **dos o más relaciones con la misma tabla**, EF Core no puede identificar automáticamente qué propiedad corresponde a cada relación.

- *Escenario:* Un sistema donde un registro tiene un "Creador" y un "Asignado", y ambos son de la tabla de usuarios. Es obligatorio usar Fluent API para especificar las claves foráneas de forma explícita.

#### 2. Evitar Conflictos de Borrado en Cascada (SQL Server)

Por defecto, EF Core configura borrados en cascada (Cascade). Sin embargo, en **SQL Server**, si hay múltiples rutas de borrado que convergen en una misma tabla, la base de datos lanzará un error de integridad referencial.

- La Fluent API permite cambiar el comportamiento a DeleteBehavior.Restrict o NoAction, lo cual es vital en sistemas complejos para que la base de datos sea estable.

### 3. Claves Primarias Compuestas

Si una tabla requiere una clave primaria formada por dos o más columnas (común en tablas de unión o registros históricos), no se puede hacer mediante atributos. Se debe usar obligatoriamente en el `OnModelCreating`:

```
builder.Entity<MiClase>().HasKey(x => new { x.IdUno, x.IdDos });
```

### 4. Mantener el Modelo "Limpio" (Separación de Conceptos)

Para seguir principios de **Clean Code**, muchos desarrolladores prefieren que las clases en la carpeta `/Models` sean objetos puros de C# (POCO) sin dependencias de infraestructura. Al usar Fluent API, se elimina la necesidad de llenar las clases de atributos como `[Table]`, `[Column]` o `[MaxLength]`, centralizando todo en el archivo de contexto de datos.

### 5. Configuraciones Avanzadas de Base de Datos

- **Precisión de Decimales:** Para campos monetarios o científicos (ej. `.HasPrecision(18, 2)`).
- **Filtros Globales de Consulta:** Muy útiles para implementar "Borrado Lógico" (Soft Delete). Puedes definir que, por defecto, el sistema ignore todos los registros que tengan una propiedad `IsDeleted = true` sin tener que recordarlo en cada consulta LINQ.
- **Conversiones de Valor:** Si deseas que un `Enum` se guarde en la base de datos como una cadena de texto (`string`) en lugar de un número entero.

### 6. Descripción del Código

Al trabajar con versiones modernas como **.NET 10** y **C# 14**, el uso de *Primary Constructors* y miembros requeridos (`required`) hace que el código sea más conciso. La Fluent API se integra perfectamente con estas características al permitir configurar el mapeo sin interferir con la nueva sintaxis de los constructores.

En el diseño que estamos siguiendo, la Fluent API es la que garantiza que las relaciones entre los tickets, los autores y los técnicos no generen errores de colisión en SQL Server.

Este bloque de código es el "manual de instrucciones" que le entregamos a Entity Framework Core para que entienda cómo construir las relaciones en la base de datos SQL Server.

Es necesario porque tenemos una **ambigüedad técnica**: la tabla `Tickets` apunta dos veces a la misma tabla de usuarios (`AspNetUsers`), una para el **Author** (quien reporta) y otra para el **Technician** (quien resuelve).

### Desglose paso a paso del código (¡¡Importante!!)

Consideremos el bloque del `Author`:

```
builder.Entity<Ticket>() // 1. "En la tabla de Tickets..."
    .HasOne(t => t.Author) // 2. "...cada ticket tiene UN Autor..."
    .WithMany() // 3. "...y cada Autor puede tener MUCHOS tickets asociados..."
    .HasForeignKey(t => t.AuthorId) // 4. "...usa la columna 'AuthorId' como la llave de conexión..."
    .OnDelete(DeleteBehavior.Restrict); // 5. "...y si borran al usuario, NO borres sus tickets."
```

# 1. .HasOne (...) y .WithMany ()

Aquí definimos una relación 1:N (uno a muchos).

- Un ticket pertenece a un solo autor.
- Un autor puede abrir muchos tickets a lo largo del tiempo.
- Al dejar WithMany () vacío (sin parámetros), le decimos a EF que no necesitamos tener una lista de tickets dentro de la clase ApplicationUser, manteniendo nuestro modelo de usuario limpio.

# 2. .HasForeignKey (t => t.AuthorId)

Le indicamos explícitamente a SQL Server qué columna debe actuar como "pegamento". Sin esto, EF Core podría intentar crear una columna genérica con un nombre extraño como ApplicationUserId.

# 3. .OnDelete (DeleteBehavior.Restrict) — Este es el "seguro de vida" para los datos.

- **El Problema:** Por defecto, SQL Server usa `Cascade Delete` (Borrado en Cascada). Si borras un usuario, SQL intenta borrar todos sus tickets automáticamente.
- **El Conflicto:** Cuando tienes dos relaciones a la misma tabla (Author y Technician), SQL Server se bloquea porque hay **múltiples caminos de borrado**. No sabe qué camino seguir y lanza un error al intentar crear la base de datos.
- **La Solución:** `Restrict` le dice a SQL: *"Si alguien intenta borrar a un usuario que tiene tickets asignados, prohíbelo"*. Esto protege la integridad referencial y evita que queden tickets "huérfanos" en el sistema.

## Resumen

Sin estas líneas de código, el comando `Update-Database` fallaría con un error críptico sobre **"cycles or multiple cascade paths"**. Al usar Fluent API, se toma el control total de la base de datos, asegurando que:

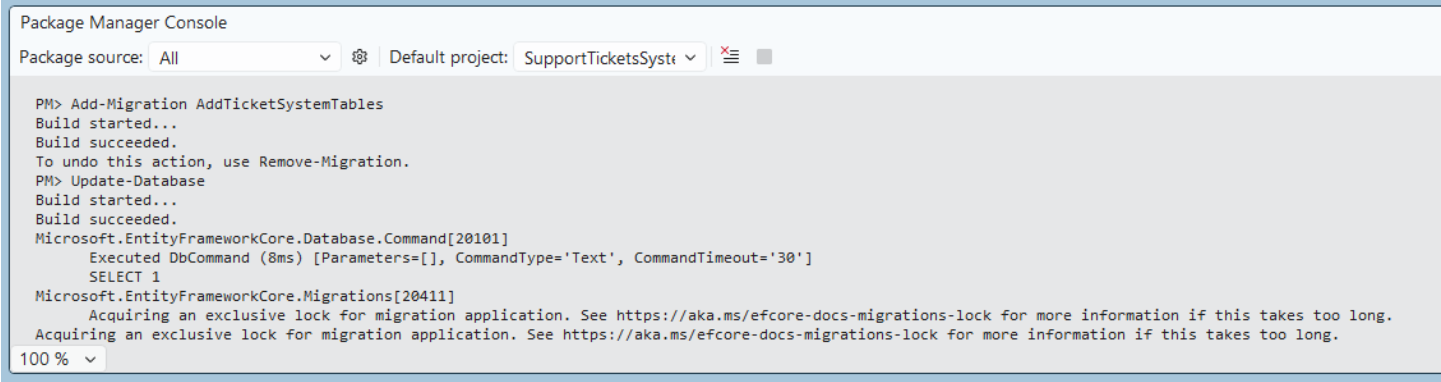
1. Las relaciones sean claras.
2. Los nombres de las columnas sean los que definimos (`AuthorId`, `TechnicianId`).
3. La base de datos sea robusta y no permita borrar información accidentalmente.

# 4. Aplicar la Migración a SQL Server

Con los modelos definidos, se va a generar las tablas en la base de datos:

1. Abrir **Package Manager Console**.
2. Ejecutar los siguientes comandos para crear la migración:

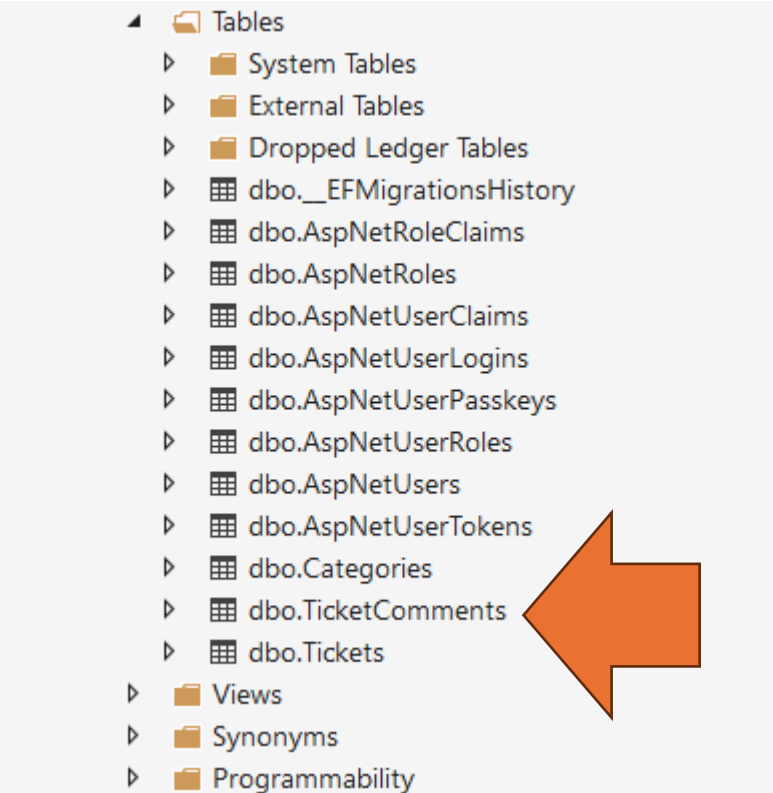
`Add-Migration AddTicketSystemTables`  
`Update-Database`





# Resultado Final del Módulo

Si revisas el **SQL Server Object Explorer**, verás que tu base de datos `SupportTicketDB` ahora tiene nuevas tablas: `Tickets`, `Categories` y `TicketComments`, con sus respectivas llaves foráneas apuntando a la tabla de usuarios.



## Actualización del Repositorio

```
PS C:\Users\luisd\source\repos\SupportTicketsSystem2026> git status
On branch main
Your branch is up to date with 'origin/main'.

Changes not staged for commit:
  (use "git add <file>..." to update what will be committed)
  (use "git restore <file>..." to discard changes in working directory)
    modified:   SupportTicketsSystem2026/Data/ApplicationDbContext.cs
    modified:   SupportTicketsSystem2026/Data/Migrations/ApplicationDbContextModelSnapshot.cs
    modified:   SupportTicketsSystem2026/Enums/TicketPriority.cs
    modified:   SupportTicketsSystem2026/Enums/TicketStatus.cs
    modified:   SupportTicketsSystem2026/Models/Category.cs
    modified:   SupportTicketsSystem2026/Models/Ticket.cs
    modified:   SupportTicketsSystem2026/Models/TicketComment.cs

Untracked files:
  (use "git add <file>..." to include in what will be committed)
    SupportTicketsSystem2026/Data/Migrations/20260319043941_AddTicketSystemTables.Designer.cs
    SupportTicketsSystem2026/Data/Migrations/20260319043941_AddTicketSystemTables.cs

no changes added to commit (use "git add" and/or "git commit -a")
PS C:\Users\luisd\source\repos\SupportTicketsSystem2026>
PS C:\Users\luisd\source\repos\SupportTicketsSystem2026> git add .
PS C:\Users\luisd\source\repos\SupportTicketsSystem2026> git status
On branch main
Your branch is up to date with 'origin/main'.

Changes to be committed:
  (use "git restore --staged <file>..." to unstage)
    modified:   SupportTicketsSystem2026/Data/ApplicationDbContext.cs
    new file:   SupportTicketsSystem2026/Data/Migrations/20260319043941_AddTicketSystemTables.Designer.cs
    new file:   SupportTicketsSystem2026/Data/Migrations/20260319043941_AddTicketSystemTables.cs
    modified:   SupportTicketsSystem2026/Data/Migrations/ApplicationDbContextModelSnapshot.cs
    modified:   SupportTicketsSystem2026/Enums/TicketPriority.cs
    modified:   SupportTicketsSystem2026/Enums/TicketStatus.cs
    modified:   SupportTicketsSystem2026/Models/Category.cs
    modified:   SupportTicketsSystem2026/Models/Ticket.cs
    modified:   SupportTicketsSystem2026/Models/TicketComment.cs
PS C:\Users\luisd\source\repos\SupportTicketsSystem2026> git commit -m "Implementando ApplicationDbContext/migracin... MÓDULO 3"
[main 837fd72] Implementando ApplicationDbContext/migracin... MÓDULO 3
 9 files changed, 957 insertions(+), 118 deletions(-)
 create mode 100644 SupportTicketsSystem2026/Data/Migrations/20260319043941_AddTicketSystemTables.Designer.cs
 create mode 100644 SupportTicketsSystem2026/Data/Migrations/20260319043941_AddTicketSystemTables.cs
PS C:\Users\luisd\source\repos\SupportTicketsSystem2026> git push origin main
Enumerating objects: 29, done.
Counting objects: 100% (29/29), done.
Delta compression using up to 24 threads
Compressing objects: 100% (16/16), done.
Writing objects: 100% (16/16), 4.02 KiB | 1.34 MiB/s, done.
Total 16 (delta 4), reused 0 (delta 0), pack-reused 0 (from 0)
remote: Resolving deltas: 100% (4/4), completed with 3 local objects.
To https://github.com/siciuprp2020/SupportTicketsSystem2026.git
 8ccf996..837fd72  main -> main
PS C:\Users\luisd\source\repos\SupportTicketsSystem2026>
```

---

### Checkpoint (Agile)

- [x] Modelado de datos completado en inglés.
- [x] Uso de Enums para control de estado y prioridad.
- [x] Relación establecida entre el sistema de tickets y el sistema de usuarios (Identity).
- [x] Base de datos actualizada con la nueva estructura de negocio.